

Gebze Technical University

CSE344 SYSTEM PROGRAMMING

-

FINAL REPORT

Veysel Cemaloğlu

200104004107

Compiling and Running

Compile the code:

```
make
```

Run the program on server side:

```
./PideShop <portNumber> <CookThreadPoolSize> <DeliveryPoolSize>  
<deliverySpeed>
```

Run the program on client side:

```
./HungryVeryMuch <portNumber> <ipAdress> <numberOfClients>  
<townSizeX> <townSizeY>
```

Clean the object files and log file:

```
make clean
```

Problem Overview

The task is to create a multithreaded C program that simulates a pizzeria shop with a special pizza oven, delivery personnel, and numerous hungry customers. The program must efficiently handle orders, cooking, and deliveries, while ensuring thread safety and proper synchronization. Additionally, it must handle signals for orderly termination and maintain a log file of activities.

To achieve these goals, the program utilizes several synchronization mechanisms, including mutexes, condition variables, and semaphores:

- **Mutexes and Condition Variables:** Used to manage shared resources and ensure synchronization between threads, preventing data races and ensuring safe access to shared data structures like the order queue.
- **Semaphores:** Used to control access to limited resources like oven slots and apparatus.

By combining these synchronization primitives, the program can efficiently manage multiple threads, handle various stages of order processing, and respond appropriately to system signals, ensuring a robust and high-performance pizzeria shop simulation.

PROBLEM SOLVING APPROACH

1. Order Handling

The program initializes orders and handles their lifecycle from reception to delivery. Each order contains details such as ID, address, shop location, and status. Orders are managed in a synchronized manner to ensure consistency and correctness.

2. Thread Management

The program uses separate thread pools for cooks and delivery personnel:

- **Cook Threads:** Responsible for preparing and cooking orders. They use semaphores to manage access to the oven and apparatus.
- **Delivery Threads:** Responsible for delivering cooked orders to customers. They manage their capacity and delivery speed to optimize delivery times.
- **Client Handler Thread (Manager):** A dedicated thread handles incoming client connections and manages the reception and processing of orders from multiple clients.

3. Mutexes and Condition Variables

Mutexes and condition variables are extensively used to ensure thread safety and synchronization:

- **Order Mutex:** Protects access to the order queue and ensures only one thread can modify it at a time.
- **Log Mutex:** Ensures that log messages are written sequentially without corruption.
- **Order Condition Variable:** Used to signal cook threads when new orders are available for processing.

4. Semaphores

Semaphores control access to limited resources:

- **Oven Semaphore:** Limits the number of concurrent orders that can be cooked in the oven.
- **Apparatus Semaphore:** Manages access to the cooking apparatus needed to place orders in the oven.

5. Signal Handling

The program includes signal handlers to manage termination signals (e.g., SIGINT):

- **Signal Handling:** When a termination signal is received, the signal handler sets a flag indicating the program should terminate. This flag is checked by all threads to ensure they exit gracefully.
- **Logging:** The signal handler ensures that all activities are properly logged before termination.

6. Server initialization

The server initializes and listens for incoming client connections. It handles order reception, processing, and logging:

- **Socket Initialization:** Sets up the server socket to listen for connections.
- **Connection Handling:** Accepts client connections and processes orders using the cook and delivery thread pools.

7. Client Order Generation

The client program generates random orders based on the number of clients and town size parameters. It sends these orders to the server for processing:

- **Random Order Generation:** Uses random number generators to create orders with varying addresses within the specified town dimensions.
- **Order Sending:** Sends orders to the server and handles responses.

TEST CASES:

Test 1:

server side: ./PideShop 8080 4 6 1

client side: ./HungryVeryMuch 8080 127.0.1.1 50 10 20

```
veysel@veysel-Yoga-Slim-7-14ARE05: ~/D... x  veysel@veysel-Yoga-Slim-7-14ARE05: ~/D... x v
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/systemprog/Final$ make
gcc -pthread -Wall -Wextra -o PideShop PideShop.c -lm
gcc -pthread -Wall -Wextra -o HungryVeryMuch HungryVeryMuch.c
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/systemprog/Final$ ./PideShop 8080 4
6 1
> PideShop active waiting for connection ...
Server IP Address: 127.0.1.1
> > Accepted connection from client PID 12597
> > 50 new customers.. Serving
> > done serving client @ XXX PID 12597
> > active waiting for connections ..
|
```

```
veysel@veysel-Yoga-Slim-7-14ARE05: ~/D... x  veysel@veysel-Yoga-Slim-7-14ARE05: ~/D... x v
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/systemprog/Final$ ./HungryVeryMuch 8
080 127.0.1.1 50 10 20
> > Client PID: 12597...
> > ...
> > All customers served
> > log file written ..
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/systemprog/Final$
```

log file overview (not all of its content):

```
1  PideShop Log Initialized
2  PideShop active, waiting for connections...
3  Accepted connection from client PID 12597
4  Received order 1 from client, address_x=0, address_y=2
5  Cook 1 preparing order 1
6  Received order 2 from client, address_x=6, address_y=1
7  Cook 2 preparing order 2
8  Received order 3 from client, address_x=8, address_y=13
9  Cook 3 preparing order 3
10 Order 1 is in the oven
11 Received order 4 from client, address_x=8, address_y=16
12 Order 2 is in the oven
13 Cook 4 preparing order 4
14 Cook 1 finished order 1
15 Received order 5 from client, address_x=7, address_y=4
16 Delivery 1 delivered order 1
17 Cook 1 preparing order 5
18 Cook 2 finished order 2
19 Received order 6 from client, address_x=9, address_y=10
20 Received order 7 from client, address_x=7, address_y=4
21 Received order 8 from client, address_x=6, address_y=7
22 Received order 9 from client, address_x=1, address_y=19
23 Order 4 is in the oven
24 Delivery 1 delivered order 3
25 Cook 2 preparing order 6
26 Cook 4 finished order 4
27 Cook 4 preparing order 7
28 Delivery 2 delivered order 6
29 Received order 10 from client, address_x=1, address_y=17
30 Received order 11 from client, address_x=7, address_y=15
31 Received order 12 from client, address_x=1, address_y=9
```

```
82 Cook 4 finished order 7
83 Cook 4 preparing order 11
84 Delivery 5 delivered order 9
85 Cook 2 finished order 9
86 Cook 2 preparing order 12
87 Delivery 5 delivered order 11
88 Cook 1 preparing order 13
89 Order 11 is in the oven
90 Cook 3 finished order 3
91 Cook 3 preparing order 14
92 Delivery 5 delivered order 5
93 Order 12 is in the oven
94 Cook 4 finished order 11
95 Cook 4 preparing order 15
96 Order 13 is in the oven
97 Order 14 is in the oven
98 Cook 2 finished order 12
99 Cook 2 preparing order 16
100 Delivery 3 delivered order 14
101 Order 16 is in the oven
102 Cook 3 finished order 14
103 Cook 3 preparing order 18
104 Delivery 6 delivered order 16
105 Order 15 is in the oven
106 Cook 1 finished order 13
107 Cook 1 preparing order 19
108 Delivery 6 delivered order 15
109 Order 18 is in the oven
110 Cook 2 finished order 16
111 Cook 2 preparing order 21
112 Order 19 is in the oven
113 Delivery 6 delivered order 19
114 Cook 4 finished order 15
115 Delivery 2 delivered order 18
116 Cook 3 finished order 18
117 Cook 3 preparing order 23
```

Test 2:

adding another client while server running:

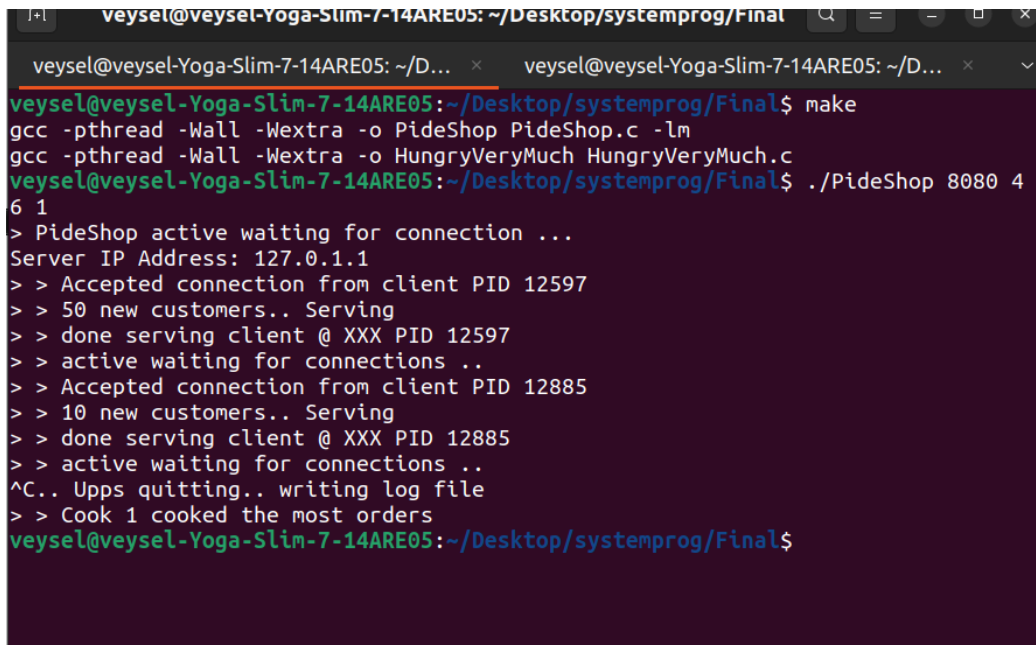
`./HungryVeryMuch 8080 127.0.1.1 10 10 20`

```
veysel@veysel-Yoga-Slim-7-14ARE05: ~/D... x  veysel@veysel-Yoga-Slim-7-14ARE05: ~/D... x v
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/systemprog/Final$ make
gcc -pthread -Wall -Wextra -o PideShop PideShop.c -lm
gcc -pthread -Wall -Wextra -o HungryVeryMuch HungryVeryMuch.c
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/systemprog/Final$ ./PideShop 8080 4
6 1
> PideShop active waiting for connection ...
Server IP Address: 127.0.1.1
> > Accepted connection from client PID 12597
> > 50 new customers.. Serving
> > done serving client @ XXX PID 12597
> > active waiting for connections ..
> > Accepted connection from client PID 12885
> > 10 new customers.. Serving
> > done serving client @ XXX PID 12885
> > active waiting for connections ..
```

```
veysel@veysel-Yoga-Slim-7-14ARE05: ~/Desktop/systemprog/Final Q ≡ − □ ×
veysel@veysel-Yoga-Slim-7-14ARE05: ~/D... x  veysel@veysel-Yoga-Slim-7-14ARE05: ~/D... x v
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/systemprog/Final$ ./HungryVeryMuch 8
080 127.0.1.1 50 10 20
> > Client PID: 12597...
> > ...
> > All customers served
> > log file written ..
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/systemprog/Final$ ./HungryVeryMuch 8
080 127.0.1.1 10 10 20
> > Client PID: 12885...
> > ...
> > All customers served
> > log file written ..
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/systemprog/Final$ |
```


Test 3:

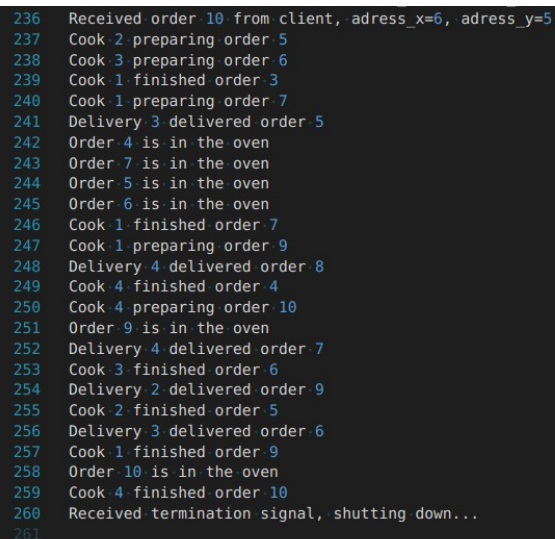
ctrl^C on server.



```
veysel@veysel-Yoga-Slim-7-14ARE05: ~/Desktop/systemprog/Final
veysel@veysel-Yoga-Slim-7-14ARE05: ~/D... x  veysel@veysel-Yoga-Slim-7-14ARE05: ~/D... x v
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/systemprog/Final$ make
gcc -pthread -Wall -Wextra -o PideShop PideShop.c -lm
gcc -pthread -Wall -Wextra -o HungryVeryMuch HungryVeryMuch.c
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/systemprog/Final$ ./PideShop 8080 4
6 1
> PideShop active waiting for connection ...
Server IP Address: 127.0.1.1
> > Accepted connection from client PID 12597
> > 50 new customers.. Serving
> > done serving client @ XXX PID 12597
> > active waiting for connections ..
> > Accepted connection from client PID 12885
> > 10 new customers.. Serving
> > done serving client @ XXX PID 12885
> > active waiting for connections ..
^C.. Upps quitting.. writing log file
> > Cook 1 cooked the most orders
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/systemprog/Final$
```

As you can see after server is terminated the most successful cook is printed on screen which will be promoted.

Meanwhile log file (end of log file after ctrl^C on server side):



```
236 Received order 10 from client, adress_x=6, adress_y=5
237 Cook 2 preparing order 5
238 Cook 3 preparing order 6
239 Cook 1 finished order 3
240 Cook 1 preparing order 7
241 Delivery 3 delivered order 5
242 Order 4 is in the oven
243 Order 7 is in the oven
244 Order 5 is in the oven
245 Order 6 is in the oven
246 Cook 1 finished order 7
247 Cook 1 preparing order 9
248 Delivery 4 delivered order 8
249 Cook 4 finished order 4
250 Cook 4 preparing order 10
251 Order 9 is in the oven
252 Delivery 4 delivered order 7
253 Cook 3 finished order 6
254 Delivery 2 delivered order 9
255 Cook 2 finished order 5
256 Delivery 3 delivered order 6
257 Cook 1 finished order 9
258 Order 10 is in the oven
259 Cook 4 finished order 10
260 Received termination signal, shutting down...
261
```

Test 4:

Wrong port number entry and if user enters arguments wrongly or missing arguments:

```
veysel@veysel-Yoga-Slim-7-14ARE05: ~/Desktop/systemprog/Final$ ./HungryVeryMuch 8081 127.0.1.1 10 10 20
> > Client PID: 13207...
> > ...
Connection failed: Connection refused
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/systemprog/Final$ ./HungryVeryMuch 8083 127.0.1.1 10 10 20
> > Client PID: 13208...
> > ...
Connection failed: Connection refused
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/systemprog/Final$ ./HungryVeryMuch 8083 127.0.1.1 10 20
Usage: ./HungryVeryMuch [portnumber] [ipAddress] [numberOfClients] [p] [q]
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/systemprog/Final$ ./HungryVeryMuch 8083 10 10 20
Usage: ./HungryVeryMuch [portnumber] [ipAddress] [numberOfClients] [p] [q]
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/systemprog/Final$ |
```

since no connection established nothing happened on server side:

```
veysel@veysel-Yoga-Slim-7-14ARE05: ~/Desktop/systemprog/Final$ ./PideShop 8080 46 1
> PideShop active waiting for connection ...
Server IP Address: 127.0.1.1
```

Test 5:

Another run case like:

server side: ./PideShop 8080 10 8 2

client side: /HungryVeryMuch 8080 127.0.1.1 100 10 20

```
veysel@veysel-Yoga-Slim-7-14ARE05: ~/D... x  veysel@veysel-Yoga-Slim-7-14ARE05: ~/D... x v
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/systemprog/Final$ ./HungryVeryMuch 8
080 127.0.1.1 100 10 20
> > Client PID: 13365...
> > ...
> > All customers served
> > log file written ..
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/systemprog/Final$
```

```
veysel@veysel-Yoga-Slim-7-14ARE05: ~/D... x  veysel@veysel-Yoga-Slim-7-14ARE05: ~/D... x v
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/systemprog/Final$ make clean
rm -f PideShop HungryVeryMuch pideshop.log
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/systemprog/Final$ make
gcc -pthread -Wall -Wextra -o PideShop PideShop.c -lm
gcc -pthread -Wall -Wextra -o HungryVeryMuch HungryVeryMuch.c
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/systemprog/Final$ ./PideShop 8080 10
8 2
> PideShop active waiting for connection ...
Server IP Address: 127.0.1.1
> > Accepted connection from client PID 13365
> > 100 new customers.. Serving
> > done serving client @ XXX PID 13365
> > active waiting for connections ..
|
```

Meanwhile log file (some snippets after this test case):

```
pidshop.log
1  PidShop Log Initialized
2  PidShop active, waiting for connections...
3  Accepted connection from client PID 13365
4  Received order 1 from client, adress_x=2, adress_y=4
5  Cook 1 preparing order 1
6  Received order 2 from client, adress_x=5, adress_y=14
7  Cook 2 preparing order 2
8  Received order 3 from client, adress_x=9, adress_y=0
9  Cook 3 preparing order 3
10 Order 1 is in the oven
11 Received order 4 from client, adress_x=4, adress_y=9
12 Cook 4 preparing order 4
13 Order 2 is in the oven
14 Cook 1 finished order 1
15 Received order 5 from client, adress_x=0, adress_y=2
16 Cook 5 preparing order 5
17 Order 3 is in the oven
18 Cook 2 finished order 2
19 Received order 6 from client, adress_x=1, adress_y=16
20 Cook 6 preparing order 6
21 Order 4 is in the oven
22 Cook 3 finished order 3
23 Received order 7 from client, adress_x=8, adress_y=19
24 Cook 7 preparing order 7
25 Order 5 is in the oven
26 Cook 4 finished order 4
27 Received order 8 from client, adress_x=2, adress_y=3
28 Cook 8 preparing order 8
29 Order 6 is in the oven
30 Cook 5 finished order 5
31 Received order 9 from client, adress_x=2, adress_y=12
32 Cook 4 preparing order 9
33 Order 7 is in the oven
34 Cook 6 finished order 6
35 Received order 10 from client, adress_x=2, adress_y=0
36 Cook 10 preparing order 10
37 Order 8 is in the oven
38 Cook 7 finished order 7
39 Received order 11 from client, adress_x=2, adress_y=1
40 Order 9 is in the oven
41 Cook 3 preparing order 11
42 Delivery 1 delivered order 1
43 Delivery 1 delivered order 2
44 Delivery 1 delivered order 3
```

```
pideshop.log
414 Cook 6 preparing order 95
415 Delivery 5 delivered order 92
416 Order 86 is in the oven
417 Order 88 is in the oven
418 Order 87 is in the oven
419 Cook 10 finished order 82
420 Cook 10 preparing order 96
421 Delivery 1 delivered order 94
422 Cook 4 finished order 86
423 Cook 4 preparing order 97
424 Delivery 4 delivered order 96
425 Cook 3 finished order 88
426 Cook 3 preparing order 98
427 Delivery 8 delivered order 97
428 Order 89 is in the oven
429 Cook 5 finished order 87
430 Cook 5 preparing order 99
431 Delivery 2 delivered order 98
432 Order 91 is in the oven
433 Order 92 is in the oven
434 Order 90 is in the oven
435 Cook 7 finished order 89
436 Cook 7 preparing order 100
437 Delivery 8 delivered order 99
438 Order 95 is in the oven
439 Cook 1 finished order 90
440 Delivery 6 delivered order 100
441 Order 97 is in the oven
442 Cook 6 finished order 95
443 Cook 8 finished order 92
444 Order 96 is in the oven
445 Cook 9 finished order 91
446 Order 94 is in the oven
447 Cook 4 finished order 97
448 Order 98 is in the oven
449 Cook 10 finished order 96
450 Order 99 is in the oven
451 Cook 2 finished order 94
452 Order 100 is in the oven
453 Cook 3 finished order 98
454 Cook 5 finished order 99
455 Cook 7 finished order 100
456 Received termination signal, shutting down...
457
```

CONCLUSION

By using a combination of multithreading, synchronization primitives, and signal handling, the program efficiently simulates a pidi shop with order processing, cooking, and delivery. The use of mutexes, condition variables, and semaphores ensures thread safety and coordination, while signal handling provides robustness for orderly termination. This approach leverages the strengths of multithreading to achieve high throughput and robustness in the simulation of a pidi shop operation.